

RTL TO GDSII Flow: MAC Unit

Arihant Agarwal

March 2026

1 RTL Design and Verification

1.1 Project Setup

Terminal Commands: Create the project directory and adjust the bash prompt for clarity.

```
mkdir mac
cd mac
export PS1="\W $ "
```

1.2 RTL Implementation (Asynchronous Reset)

Verilog Code: Creating the initial Multiplier-Accumulator (MAC) module.

```
gvim mac.v

1 module mac #(parameter n=32) (
2     input wire clk,
3     input wire rst,
4     input wire en,
5     input wire clr,
6     input wire signed [n-1:0] a,
7     input wire signed [n-1:0] b,
8     output reg signed [2*n-1:0] result,
9     output reg valid
10 );
11
12 always @(posedge clk or posedge rst) begin
13     if (rst) begin
14         result <= 0;
15         valid <= 1'b0;
16     end
17     else if (clr) begin
18         result <= 0;
19         valid <= 1'b0;
20     end
21     else if (en) begin
22         result <= result + (a*b);
23         valid <= 1'b1;
24     end
25     else begin
26         valid <= 1'b0;
27     end
28 end
29 endmodule
```

Listing 1: mac.v

1.3 Testbench Creation

Verilog Code: Writing the testbench to verify the MAC functionality.

```
gvim tb_mac.v

1  `timescale 1ns/1ps
2
3  module tb_mac;
4
5  parameter n = 32;
6  reg clk;
7  reg rst;
8  reg en;
9  reg clr;
10 reg signed [n-1:0] a;
11 reg signed [n-1:0] b;
12 wire signed [2*n-1:0] result;
13 wire valid;
14
15 mac #(n) dut(
16     .clk(clk),
17     .rst(rst),
18     .en(en),
19     .clr(clr),
20     .a(a),
21     .b(b),
22     .result(result),
23     .valid(valid)
24 );
25
26 always #5 clk = ~clk;
27
28 initial begin
29     $dumpfile("mac.vcd");
30     $dumpvars(0, tb_mac);
31     $sdf_annotate("mac.sdf", dut);
32     clk = 0; rst = 1; en = 0; clr = 0; a = 0; b = 0;
33     #10;
34     rst = 0;
35     #10;
36
37     //Test 1
38     clr = 1;
39     #10;
40     clr = 0;
41     en = 1;
42     a = 3;
43     b = 4;
44     #10;
45     a = 2;
46     b = 5;
47     #10;
48     en = 0;
49     #10
50     $display("You got, %d ", result);
51     if (result == 22)
52         $display("Pass \n");
53     #10;
54
55     //Test 2
56     clr = 1;
57     #10;
58     clr = 0;
59     en = 1;
60     a = 1;
61     b = 1;
62     #10;
63     en = 0;
```

```

64     #10
65     $display("You got, %d ", result);
66     if (result == 1)
67         $display("Pass \n");
68     #10;
69
70     //Test 3
71     clr = 1;
72     #10;
73     clr = 0;
74     en = 1;
75     a = -1;
76     b = -5;
77     #10;
78     en = 0;
79     #10
80     $display("You got, %d ", result);
81     if (result == 5)
82         $display("Pass \n");
83     #10;
84
85     $finish;
86 end
87
88 endmodule

```

Listing 2: tb_mac.v

1.4 Behavioral Simulation

Terminal Commands: Compiling and running the simulation using Icarus Verilog and GTKWave.

```

iverilog mac.v tb_mac.v
vvp a.out
gtkwave mac.vcd

```

2 Logic Synthesis

2.1 Synthesis Script

Yosys Script: Preparing the script to map the RTL to the OSU 0.18 μ m standard cell library.

```
gvim mac.ys
```

```

1 read_verilog mac.v
2 synth -top mac
3 dfflibmap -liberty /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
4 abc -liberty /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
5 write_verilog synth_mac.v
6 stat -liberty /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib

```

Listing 3: mac.ys

Terminal Command: Executing Yosys.

```
yosys mac.ys
```

2.2 Synthesis Statistics

Terminal Output: Gate count and area report generated by Yosys.

```

=== mac ===
Number of wires:          14832
Number of wire bits:      15020

```

```

Number of public wires:      8
Number of public wire bits: 133
Number of memories:         0
Number of memory bits:      0
Number of processes:        0
Number of cells:            7457
  AND2X1                    122
  AOI21X1                   298
  AOI22X1                   40
  DFFSR                     65
  INVX1                     520
  MUX2X1                    1
  NAND2X1                   1920
  NAND3X1                   278
  NOR2X1                    575
  NOR3X1                    6
  OAI21X1                   1054
  OAI22X1                   32
  OR2X1                    108
  XNOR2X1                  1166
  XOR2X1                   1272

Chip area for module 'mac': 270626.000000

```

Yosys Synthesis Statistics

2.3 Post-Synthesis Simulation

Terminal Commands: Simulating the generated gate-level netlist.

```

iverilog -Wall -gspecify /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.v
synth_mac.v tb_mac.v
vvp a.out
gtkwave mac.vcd

```

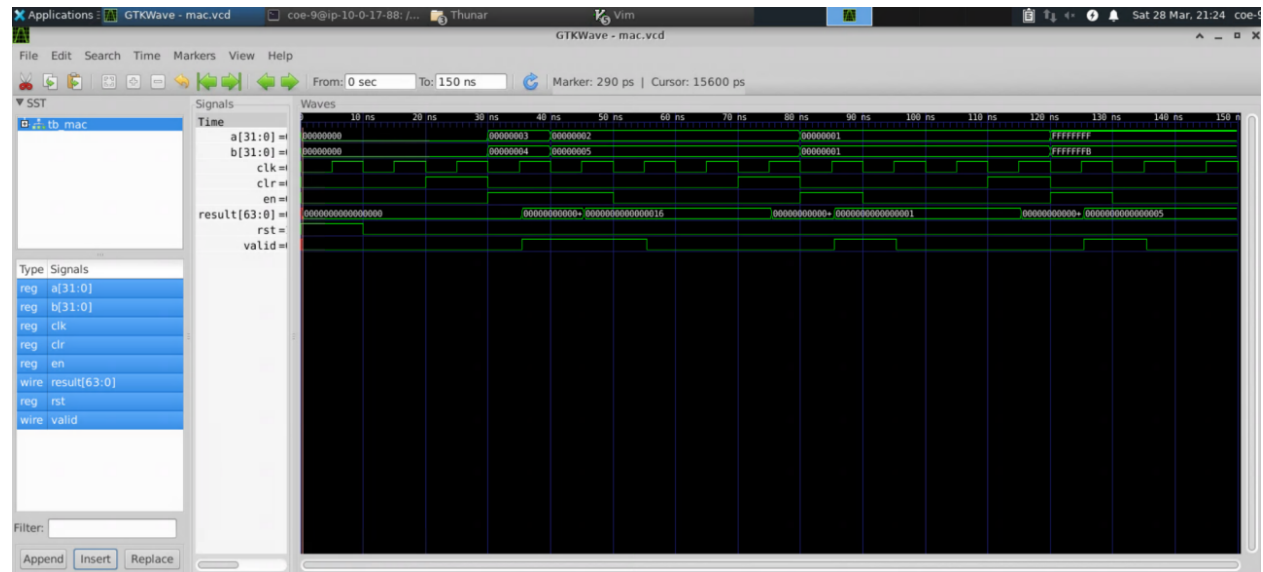


Figure 1: Post-Synthesis Simulation Waveforms

3 Static Timing Analysis (STA)

3.1 Timing Constraints

SDC File: Defining clock and I/O delays.

```
gvim mac.sdc
```

```
1 create_clock -name clk -period 11 [get_ports clk]
2 set_input_delay 1 -clock clk [get_ports {a b en clr}]
3 set_output_delay 1 -clock clk [get_ports {result valid}]
4 set_false_path -from [get_ports rst]
```

Listing 4: mac.sdc

3.2 STA Script

TCL Script: OpenSTA commands to report timing and generate the SDF file.

```
gvim mac_sta.tcl
```

```
1 read_liberty /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib
2 read_verilog synth_mac.v
3 link_design mac
4 read_sdc mac.sdc
5 report_checks -path_delay max
6 report_checks -path_delay min
7 report_worst_slack
8 report_tns
9 write_sdf mac.sdf
```

Listing 5: mac_sta.tcl

Terminal Command: Running OpenSTA.

```
sta mac_sta.tcl
```

3.3 Timing Reports

Terminal Output: Evaluating Setup and Hold times for different clock periods. Note that the generated mac.sdf file is annotated in the testbench using \$sdf_annotate("mac.sdf", DUT);.

```
#For 10 ns clock
Delay      Time      Description
-----
0.00       0.00      clock clk (rise edge)
0.00       0.00      clock network delay (ideal)
0.00       0.00      ~ _22216_/CLK (DFFSR)
0.29       0.29      ~ _22216_/Q (DFFSR)
0.06       0.35      v _14862_/Y (A0I21X1)
0.00       0.35      v _22216_/D (DFFSR)
0.35       0.35      data arrival time

0.00       0.00      clock clk (rise edge)
0.00       0.00      clock network delay (ideal)
0.00       0.00      clock reconvergence pessimism
0.00       0.00      ~ _22216_/CLK (DFFSR)
-0.02      -0.02      library hold time
-0.02      -0.02      data required time

-----
-0.02      data required time
-0.35      data arrival time
-----

0.37       slack (MET)

worst slack -0.21
tns -0.33
```

Timing Report: 10 ns Clock (Setup Violation)

```
#For 11 ns clock
Startpoint: a[31] (input port clocked by clk)
Endpoint: _22279_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max
```

Delay	Time	Description

... [Combinational Logic Delays] ...		
0.00	10.11	v _22279_/D (DFFSR)
	10.11	data arrival time
11.00	11.00	clock clk (rise edge)
0.00	11.00	clock network delay (ideal)
0.00	11.00	clock reconvergence pessimism
	11.00	^ _22279_/CLK (DFFSR)
-0.09	10.91	library setup time
	10.91	data required time

	10.91	data required time
	-10.11	data arrival time

	0.79	slack (MET)

Startpoint: _22216_ (rising edge-triggered flip-flop clocked by clk)		
Endpoint: _22216_ (rising edge-triggered flip-flop clocked by clk)		
Path Group: clk		
Path Type: min		

Delay	Time	Description

... [Hold Time Check] ...		

	0.37	slack (MET)

worst slack 0.79		
tns 0.00		

Timing Report: 11 ns Clock (Met)

3.4 Maximum Frequency Calculation

Theory & Calculation: Using the setup slack from the 11ns report to determine the maximum achievable operating frequency.

$$T_{min} = T_{clk} - \text{Setup Slack}$$

$$T_{min} = 11 \text{ ns} - 0.79 \text{ ns} = 10.21 \text{ ns}$$

$$F_{max} = \frac{1}{T_{min}}$$

$$F_{max} = \frac{1}{10.21 \times 10^{-9}}$$

$$F_{max} \approx 97.94 \text{ MHz}$$

Terminal Commands: Final post-STA Gate Level Simulation.

```
iverilog -gspecify /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.v synth_mac.v
tb_mac.v
vvp a.out
gtkwave mac.vcd
```

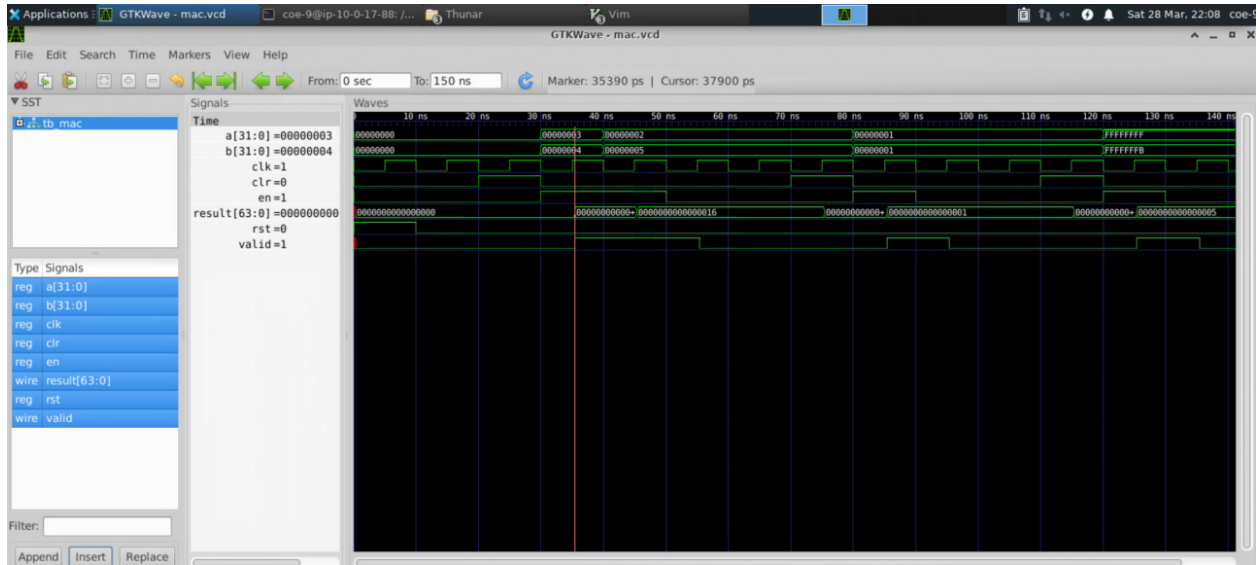


Figure 2: Gate-Level Simulation with SDF Annotation

4 Design for Testability (DFT) and ATPG

4.1 RTL Refactoring for Testability

Theory & Code: Uninitialized sequential state caused unknown ('X') propagation during Automatic Test Pattern Generation (ATPG). To resolve this, the asynchronous reset was redesigned into a synchronous reset to prevent X-propagation and make the flip-flops fully controllable.

```

1  `timescale 1ns/1ps
2
3  module mac #(parameter n=32) (
4      input wire clk,
5      input wire rst,
6      input wire en,
7      input wire clr,
8      input wire signed [n-1:0] a,
9      input wire signed [n-1:0] b,
10     output reg signed [2*n-1:0] result,
11     output reg valid
12 );
13
14 always @(posedge clk) begin
15     if (rst || clr) begin
16         result <= 0;
17         valid <= 0;
18     end
19     else if (en) begin
20         result <= result + (a*b);
21         valid <= 1;
22     end
23     else begin
24         valid <= 0;
25     end
26 end
27
28 endmodule

```

Listing 6: mac.v (Synchronous Reset Version)

4.2 Fault Synthesis and ATPG

Terminal Commands: Synthesizing the fault models, cutting the clock/reset nets, and generating test vectors while bypassing specific control signals (`en=1`, `clr=0`) to ensure the circuit is active and avoids random states.

```
fault synth --liberty /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib --top
  mac -o mac_synth.v mac.v
# Output: mac_synth.v

fault cut --clock clk --reset rst -o mac_cut.v mac_synth.v
# Output: mac_cut.v

fault atpg --cell-model /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.v --clock
  clk --reset rst --bypassing en=1 --bypassing clr=0 -o mac_atpg.json mac_cut.v
```

4.3 ATPG Generation Report

Terminal Output: Output logs demonstrating high fault coverage via partial-sequential ATPG.

```
Generating LALR tables
WARNING: 183 shift/reduce conflicts
Processing module mac...
Found 22523 fault sites in 7005 gates and 138 ports.
Performing simulations...
Initial coverage: 0.0%
Time elapsed: 107.01s.
Simulations concluded: Coverage 98.45491%
Writing raw generated test vectors in Fault JSON format to mac_atpg.json...
Finding essential test vectors...
Found 35 essential test vectors.
Performing compaction...
Initial TV Count: 100. Compacted TV Count: 61.
Successfully compacted test vectors by a ratio of 39.00%.
Writing compacted generated test vectors in Fault JSON format to mac_atpg.json...
coe-9@ip-10-0-17-88:~/test/mac$
```

Fault ATPG Output Log

5 Key Takeaways and Theory

Theoretical Principles & Conclusions:

- **Control Signal Bypassing:** Bypassing control signals during ATPG (`--bypassing en=1`, `--bypassing clr=0`) ensures that the circuit remains active and prevents it from falling into uninitialized or random states.
- **DFT Starts from RTL:** Poor RTL design directly leads to ATPG failure. Testability must be considered early in the coding phase.
- **X-Propagation is Fatal:** If the output resolves to an unknown state ('X'), the fault coverage immediately drops to 0%.
- **ATPG vs. Testbenches:** The stimulus generated by ATPG is purely structural (focused on toggling nodes to detect stuck-at faults) and differs completely from a functional testbench.
- **Combinational / Partial Sequential ATPG (Non-scan):** Performed gate-level ATPG on a MAC (osu018 PDK), resolving X-propagation and reset issues to reach **98.45%** coverage over 22,523 faults (~7,005 gates) using 61 compacted test vectors.

6 Advanced Design for Testability (DFT): Scan Chain Insertion

6.1 Automated Scan Constraints and Strategy Shift

Terminal Output: Initial automated scan insertion using `fault chain` generates both internal and boundary scans.

```
fault chain --liberty /home/coe-9/Desktop/Technology_PDKs/osu018/osu018_stdcells.lib --clock
clk --reset rst --scl-config scan_config.yaml --skip-synth -o mac_scan.v mac_synth.v
```

```
Processing file mac_synth.v...
Generating LALR tables
WARNING: 183 shift/reduce conflicts
Processing module mac...
Chaining internal flip-flops...
Internal scan chain successfully constructed. Length: 65
Boundary scan cells successfully chained. Length: 131
Total scan-chain length: 196
Done.
```

Initial Automated Scan Attempt

Terminal Commands: Inspecting the automatically generated `mac_scan.v` file to verify the inserted scan ports (`sin`, `sout`, `shift`) and review the fault metadata header.

```
grep -i sout mac_scan.v
grep -i sin mac_scan.v
grep -i shift mac_scan.v
head -100 mac_scan.v
```

Theory & Analysis: Upon inspecting the output of `head -100 mac_scan.v`, it is evident that the automated tool outputs a netlist that includes unwanted boundary scan registers alongside the internal scan chains. To achieve a purely internal scan chain architecture, we must abandon this automated output and manually insert scan multiplexers directly into the original synthesized netlist (`synth_mac.v`), which strictly contains only the 65 internal flip-flops.

File	Origin	Action
<code>synth_mac.v</code>	Yosys output (Official synthesized netlist)	USE THIS
<code>mac_synth.v</code>	Fault-generated (from <code>fault synth</code>)	Derived
<code>mac_scan.v</code>	Automated output (contains boundary scans)	Do Not Use
<code>mac_cut.v</code>	FFs removed for combinational ATPG	Do Not Use

Table 1: File Selection for Manual Scan Insertion

6.2 Manual Mux-Based Scan Insertion

Theory & Implementation: The core principle of mux-based scan insertion is converting a standard D-flip-flop data input from `.D(original.data)` to a multiplexed input: `.D(shift ? scan.in : original.data)`.

Terminal Commands: Isolating the flip-flops to plan the scan chains.

```
cp synth_mac.v mac_scan_manual.v
grep -n "DFFPOSX1" synth_mac.v > dff_list.txt
wc -l dff_list.txt
# Output: 65 dff_list.txt
grep -n "DFFPOSX1" synth_mac.v | head -20
```

Theory & Design Principles: By manually editing `mac_scan_manual.v`, a 5-chain mux-based scan architecture is implemented by partitioning the 65 flip-flops into balanced chains (length of 13 FFs each). The manual insertion requires four strict conditions:

1. **Scan Mux Insertion:** The first FF in a chain takes the primary scan input (`sin`). Example: `.D(shift ? sin[0] : _00000_[0])`

2. **Proper Chaining:** Each subsequent FF takes the output of the previous FF. Example: `.D(shift ? scan_chain0[0] : _00000_[1])`
3. **Functional Path Preservation:** Original logic must not be broken. Example: `assign result[0] = q_21971;`
4. **Clean Scan State Propagation:** Creating correct shift paths. Example: `assign scan_chain0[i] = q_2197x;`

6.3 Scan Verification Testbench

Verilog Code: Creating a testbench to verify both the scan shift mode and the functional mode of the modified netlist.

```

1  `timescale 1ns/1ps
2
3  module tb_mac_scan;
4
5  reg clk;
6  reg rst;
7  reg en;
8  reg clr;
9  reg shift;
10
11 reg signed [31:0] a, b;
12
13 reg [4:0] sin;
14 wire [4:0] sout;
15
16 wire signed [63:0] result;
17 wire valid;
18
19 // DUT
20 mac uut (
21     .clk(clk),
22     .rst(rst),
23     .en(en),
24     .clr(clr),
25     .a(a),
26     .b(b),
27     .result(result),
28     .valid(valid),
29     .shift(shift),
30     .sin(sin),
31     .sout(sout)
32 );
33
34 // Clock
35 always #5 clk = ~clk;
36
37 integer i;
38
39 initial begin
40     $dumpfile("scan.vcd");
41     $dumpvars(0, tb_mac_scan);
42
43     // Init
44     clk = 0; rst = 1; en = 0; clr = 0; shift = 0; sin = 0; a = 0; b = 0;
45
46     #20;
47     rst = 0;
48
49     // SCAN SHIFT TEST
50     shift = 1;
51     $display("---- SHIFTING START ----");
52

```

```

53 // Apply pattern
54 for (i = 0; i < 20; i = i + 1) begin
55     sin = i; // Feed different bits into 5 chains
56     #10;
57     $display("Cycle %0d | sin=%b | sout=%b", i, sin, sout);
58 end
59
60 // FUNCTIONAL TEST
61 shift = 0;
62 en = 1;
63 a = 10;
64 b = 5;
65
66 #50;
67 $display("Result = %d", result);
68
69 #50;
70 $finish;
71 end
72
73 endmodule

```

Listing 7: tb_mac_scan.v

6.4 Simulation Analysis

Terminal Commands: Running the scan testbench.

```

iverilog -o sim tb_mac_scan.v mac_scan_manual.v /home/coe-9/Desktop/Technology_PDKs/osu018/
osu018_stdcells.v
vvp sim

```

```

VCD info: dumpfile scan.vcd opened for output.
---- SHIFTING START ----
Cycle 0 | sin=00000 | sout=00000
... [Cycles 1 to 12] ...
Cycle 12 | sin=01100 | sout=00000
Cycle 13 | sin=01101 | sout=00001
Cycle 14 | sin=01110 | sout=00010
Cycle 15 | sin=01111 | sout=00011
Cycle 16 | sin=10000 | sout=00100
...
Result =      69797289155524175

```

Simulation Console Output

Theory & Analysis: The testbench confirms the dual-mode operation of the circuit:

- **Scan Mode (shift = 1):** The data applied to **sin** emerges at **sout** exactly 13 clock cycles later. Because the 65 flip-flops were divided into 5 parallel chains, each chain is exactly 13 flip-flops long ($13 \times 5 = 65$). The propagation path is strictly: **sin[i]** → **FF1** → **FF2** → ... → **FF13** → **sout[i]**.
- **Functional Mode (shift = 0):** When shift is de-asserted, the MAC resumes normal operation. The non-zero result confirms that the scan multiplexer logic did not break the original combinational data path.

Terminal Command: Viewing the waveforms.

```

gtkwave scan.vcd

```

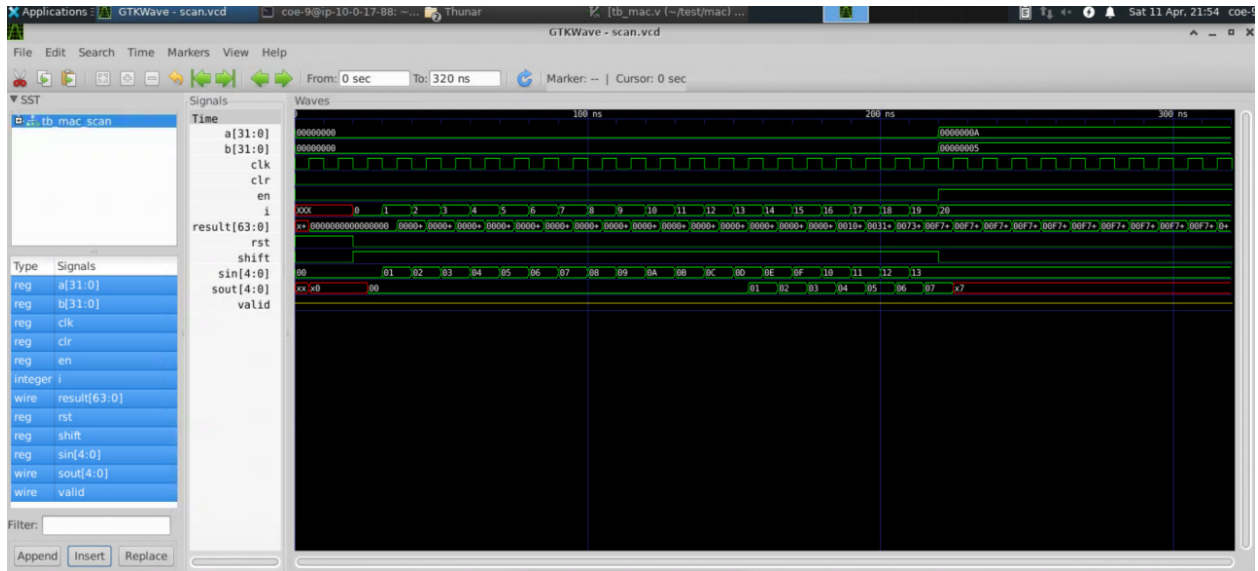


Figure 3: Scan Shift Behavior: 13-Cycle Latency Observation

6.5 ATPG on Scan-Inserted Design (Fault v0.9.4)

Terminal Commands: Cutting the sequential elements to expose the combinational logic, and running ATPG.

```
fault cut --clock clk --reset rst -o mac_scan_cut.v mac_scan_manual.v

fault atpg --clock clk --cell-model /home/coe-9/Desktop/Technology_PDKs/osu018/
osu018_stdcells.v -o mac_scan_atpg.json mac_scan_cut.v

head -20 mac_scan_atpg.json
```

Terminal Output: Coverage and compaction metrics.

```
Time elapsed: 187.58s.
Simulations concluded: Coverage 96.504166%
Writing raw generated test vectors in Fault JSON format to mac_scan_atpg.json...
Finding essential test vectors...
Found 14 essential test vectors.
Performing compaction...
Initial TV Count: 100. Compacted TV Count: 19.
Successfully compacted test vectors by a ratio of 81.00%.
Writing compacted generated test vectors in Fault JSON format to mac_scan_atpg.json...

{
  "coverageList" : [
    {
      "goldenOutput" : "",
      "vector" : [
        [
          "+",
          1
        ],
        [
          "+",
          0
        ],
        [
          "+",
          2238043705
        ],
        [
          "+",
          3531437605
        ]
      ]
    }
  ]
}
```

ATPG Generation Log

7 Workflow Summary

Flow: RTL $\rightarrow$ Synthesis $\rightarrow$ Scan Insertion $\rightarrow$ Verification $\rightarrow$ ATPG $\rightarrow$ Compaction